UNIT II
Model Validation, Visualization & Traditional Interpretable Algorithms

Model Validation — Training, Validation & Test Data

Before a machine-learning model can be trusted, it must be shown to generalize — to perform well on data it has never seen, not merely to have memorized the data it was trained on. Model validation is the set of practices used to make and check this claim.

The three-way data split:

Model development conventionally divides the available data into three parts, each with a distinct role:

- **Training data:** the portion used to fit the model's internal parameters. The model learns its patterns — coefficients, tree splits, weights — from these labelled examples.
- **Validation data:** a separate, held-out portion used during development to tune hyperparameters and select the best model configuration among several candidates, and to catch early signs of overfitting or underfitting — all without ever touching the test set.
- **Test data:** completely unseen data, used only once, at the very end, to produce a final, unbiased estimate of real-world performance. If test data is used for any tuning decision, the resulting performance estimate becomes optimistically biased and untrustworthy.

Overfitting and underfitting:

The reason this split matters is that a model can fail to generalize in two opposite ways:

- **Overfitting:** the model learns the training data too closely, including its noise and idiosyncrasies. This shows up as high training accuracy but poor performance on new data — a large gap between training and test performance.
- **Underfitting:** the model is too simple to capture the underlying pattern in the data at all, so it performs poorly on both the training data and the test data.

Good generalization is indicated when training and test performance stay close to each other. A widening gap, where training performance is much higher than test performance, is the classic symptom of overfitting; both being poor is the classic symptom of underfitting. This three-way split is the foundation for every later step covered in this unit — hyperparameter tuning, model selection, and interpretability analysis — because a reliable explanation of a model's behaviour is only meaningful if the model itself reliably generalizes.

Hyperparameters, Tuning, Model Selection & Cross-Validation

Hyperparameters

A hyperparameter is a configuration value set before training begins, rather than a value learned from the data. Common examples include k (the number of neighbours) in k-Nearest Neighbours, the maximum depth of a Decision Tree, C/kernel/gamma in a Support Vector Machine, and K (the number of clusters) in K-Means.

Hyperparameter tuning

Hyperparameter tuning is the process of systematically searching for the combination of hyperparameter values that gives the best validation performance for a given model. Three common tuning techniques are used in practice:

- **Grid Search:** exhaustively evaluates every combination from a predefined grid of candidate values — thorough, but computationally expensive as the number of hyperparameters grows.
- **Random Search:** samples random combinations from the hyperparameter space — often finds near-optimal settings much faster than an exhaustive grid search.
- **Bayesian Optimization:** builds a probabilistic model of how performance depends on the hyperparameters, and uses it to intelligently choose which configuration to try next, converging faster than random search when each evaluation is expensive.

Cross-validation

A single train/validation split can give a noisy, split-dependent estimate of performance. k-Fold Cross-Validation addresses this by dividing the training data into k equal folds, training on (k−1) folds and validating on the remaining fold, and repeating this k times so that every fold serves as the validation set exactly once. Averaging the k resulting scores gives a far more reliable estimate than a single split. This is the mechanism typically wrapped around Grid Search or Random Search, so that every candidate hyperparameter configuration is judged on its average cross-validated score rather than one lucky or unlucky split. Stratified k-Fold preserves the original class proportions in every fold and is preferred for imbalanced classification datasets.

Model selection

Model selection compares multiple candidate models (or hyperparameter configurations) using their cross-validated performance, and explicitly balances several criteria rather than optimizing accuracy alone:

- Performance — how well the model scores on the validation/cross-validation metric.
- Generalization — how small the gap is between training and validation/test performance.
- Complexity — simpler models are preferred when performance is comparable, since they are less likely to overfit and are easier to maintain.
- Interpretability — whether the model's reasoning can be understood and trusted, which matters especially in regulated or high-stakes domains.

The best model is therefore the one that jointly offers acceptable performance, good generalization, manageable complexity, and sufficient interpretability for the task at hand — not simply the model with the single highest accuracy number.

Validation Curves and Learning Curves:

Validation curves and learning curves are the two key diagnostic plots used to visually identify overfitting and underfitting, and to decide what to do about them. Both diagnose the same underlying problem — poor generalization — but vary a different axis: the validation curve varies a hyperparameter, while the learning curve varies the amount of training data.

Validation curve

A validation curve is generated by training a model repeatedly across a range of values for one hyperparameter (e.g. Decision Tree max_depth, or a regularization strength), and plotting the training score and the validation (cross-validation) score at each value.

- **Underfitting region:** at low model complexity, both training and validation scores are low and close together — the model is too simple to fit even the training data well.

- **Good-fit region:** as complexity increases, both scores rise and stay close together, with the validation score near its peak — this is the desirable operating range.
- **Overfitting region:** beyond a certain complexity, the training score keeps rising while the validation score plateaus or falls, opening a widening gap — the model is fitting noise specific to the training set.

Validation Curve – Example

A validation curve shows how the model's performance changes as a single hyperparameter varies.

Example: Decision Tree Regression – Effect of Maximum Depth

Step 1: Given Data

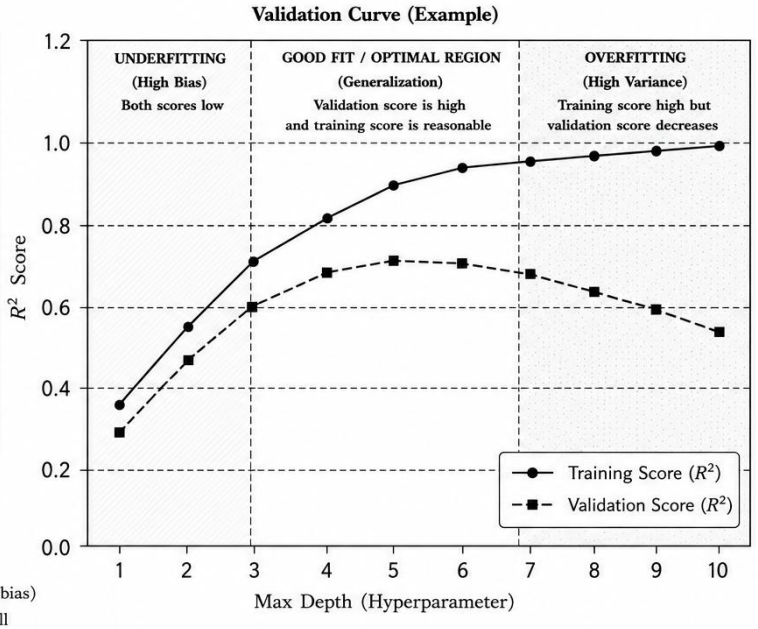
Max Depth (Hyperparameter)	Training Score (R^2)	Validation Score (R^2)
1	0.35	0.28
2	0.55	0.46
3	0.70	0.60
4	0.82	0.68
5	0.90	0.70
6	0.95	0.69
7	0.97	0.66
8	0.98	0.62
9	0.99	0.58
10	1.00	0.54

Step 2: Plot the Validation Curve

- X-axis: Hyperparameter (Max Depth)
- Y-axis: R^2 score

Step 3: Interpret the Curve

- At low depth (1–2): Both scores are low → Underfitting (high bias)
- From depth 3 to 5: Both scores increase → Model learning well
- At depth > 5: Training score keeps improving, but validation score decreases → Overfitting (high variance)
- Best depth is around 5 (highest validation score).



Use the validation curve to choose the hyperparameter value that gives the best validation performance (balances bias and variance).

Learning curve

A learning curve plots training score and validation score against the size of the training dataset, for a fixed model, revealing the model's bias-variance behaviour as more data is added.

- **High variance (overfitting):** a large, persistent gap between a high training score and a lower validation score indicates the model fits the training data much better than it generalizes — collecting more data, or adding regularization, tends to close this gap.
- **High bias (underfitting):** both curves converge to a similarly low score, with little improvement as more data is added — the model itself is too simple, and more data will not help; a more expressive model or richer features are needed instead.
- **Good fit:** both curves converge to a high score with a small remaining gap, indicating the model is appropriately complex for the amount of data available.

Learning Curve – Example

A learning curve shows how the model's performance changes as the training set size increases.

Example: Predict house price using a regression model.

Step 1: Given Data

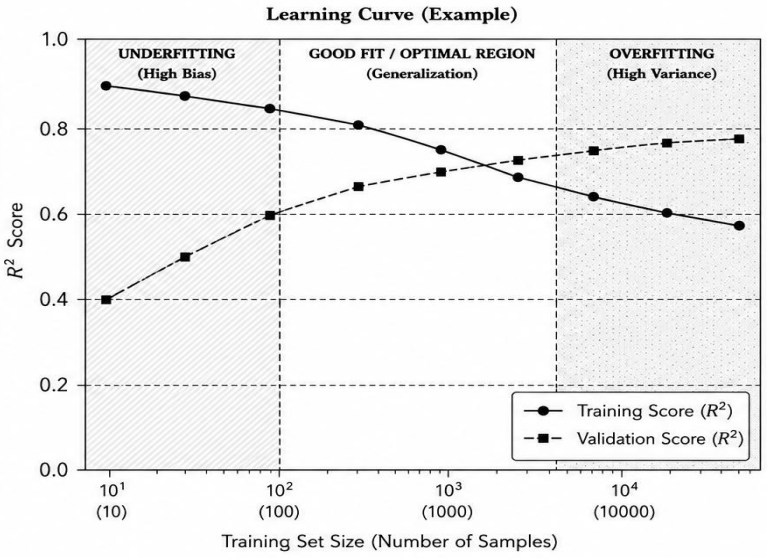
Training Set Size (Number of samples)	Training Score (R^2)	Validation Score (R^2)
50	0.95	0.40
100	0.92	0.50
200	0.88	0.60
400	0.82	0.68
800	0.75	0.72
1600	0.68	0.75
3200	0.62	0.77
6400	0.58	0.78

Step 2: Plot the Learning Curve

- X-axis: Training set size (use log scale)
- Y-axis: R^2 score

Step 3: Interpret the Curve

- Large gap at small data → High variance (overfitting)
- Gap decreases as data increases
- Curves converge → Model generalizes well
- If both curves remain low → High bias (underfitting)



Interpretation: Collecting more data helps until the model reaches the optimal region; beyond that, it may overfit.

Pattern observed	Diagnosis	Typical fix
Both scores low & close (either plot)	Underfitting (high bias)	More complex model / better features
Training score high, validation score much lower, gap not closing	Overfitting (high variance)	More data, regularization, simpler model
Both scores high and close	Good generalization	Keep current setting

Used together, the validation curve tells you where to set a hyperparameter, and the learning curve tells you whether the real fix is to collect more data or to change the model itself — a diagnosis a single accuracy number could never reveal.

Classification Model Visualization

Visual diagnostics for classifiers go beyond a single accuracy number to reveal where and how a model errs. The main tools are the confusion matrix and classification report, the ROC curve and AUC, the Precision-Recall Curve, and discrimination threshold plots.

Confusion matrix and classification report

The confusion matrix cross-tabulates predicted labels against actual labels into four cells:

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive (TP)	False Negative (FN)
Actual: Negative	False Positive (FP)	True Negative (TN)

From these four counts, every major classification metric is derived:

Accuracy = (TP+TN)/Total,

Precision = TP/(TP+FP),

Recall = TP/(TP+FN),

F1-score = the harmonic mean of Precision and Recall.

Beyond producing these numbers, the matrix reveals systematic error patterns — for example, if a model consistently confuses two specific classes, this shows up as a large off-diagonal count that a single accuracy figure would completely hide. The classification report extends this to precision, recall, F1-score, and support computed per class, which is especially useful for multi-class problems.

ROC curve and AUC

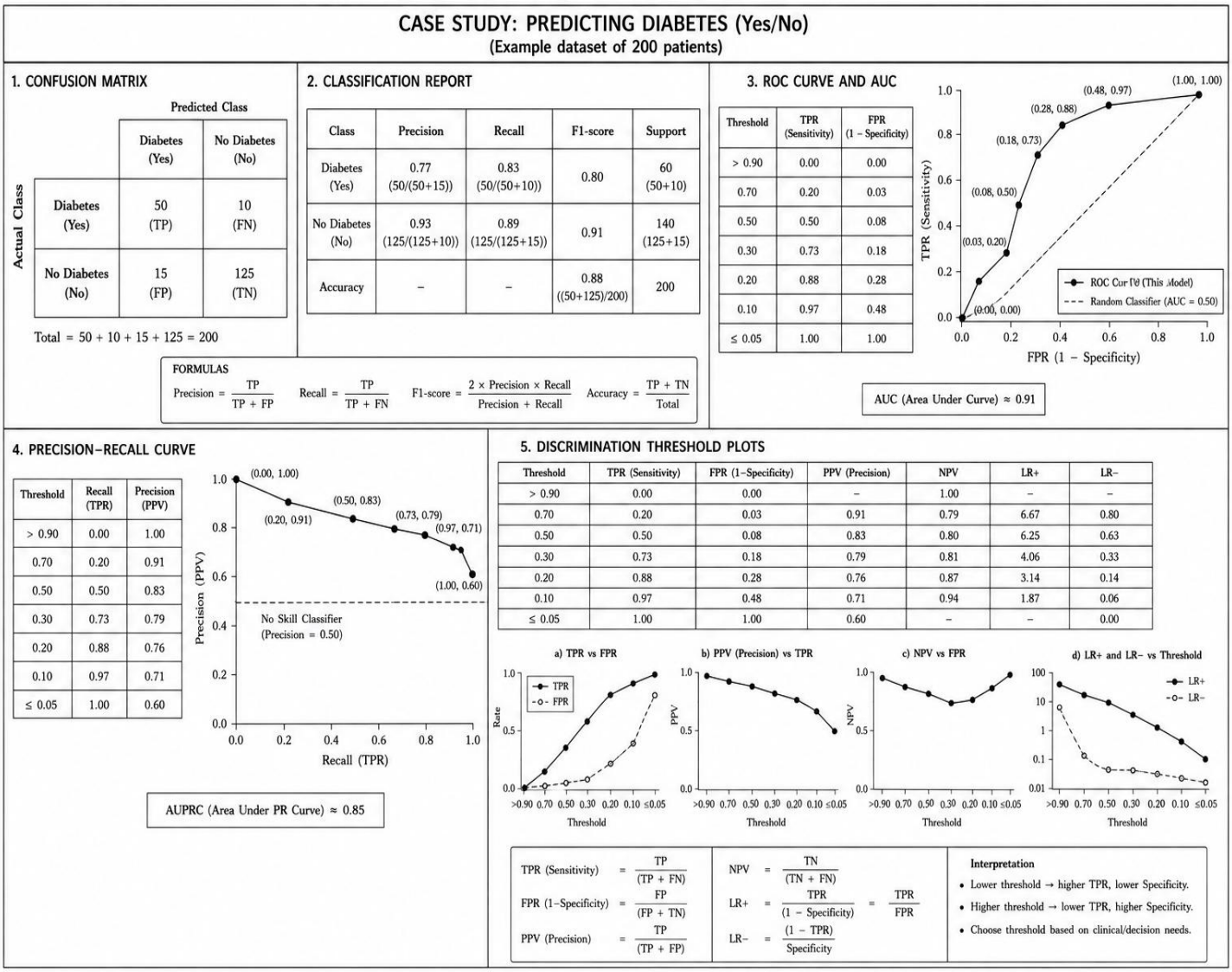
The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate against the False Positive Rate as the classification threshold is swept across its full range, with each point corresponding to a different threshold's confusion matrix. AUC (Area Under the Curve) condenses the entire curve into a single number: AUC = 1.0 is a perfect classifier, AUC = 0.5 is equivalent to random guessing, and AUC below 0.5 is worse than random. This makes ROC/AUC useful for comparing multiple candidate models' ranking ability independent of any one chosen threshold. A caveat: on severely imbalanced datasets, ROC/AUC can look misleadingly good, which is why a Precision-Recall curve is often reported alongside it in such cases.

Precision-Recall Curve (PRC)

The PRC plots precision against recall across thresholds. It is preferred over the ROC curve specifically when the dataset is highly imbalanced (e.g. fraud detection, rare-disease screening), because ROC can look overly optimistic in such cases — dominated by the large number of true negatives — while the PR curve focuses directly on performance for the minority (positive) class. A model with a large area under the PRC maintains good precision even at high recall.

Discrimination threshold plots

These plots visualize precision, recall, F1-score, and queue rate as functions of the classification threshold itself, helping practitioners choose an operating point aligned with business or domain constraints — for example, minimizing false negatives in disease screening, even at the cost of more false positives. This makes explicit the otherwise hidden decision of exactly where to draw the line between classes; raising the threshold in the spam example above would reduce false positives but increase false negatives, and the discrimination-threshold plot visualizes this trade-off directly.



Regression Model Visualization

Regression diagnostics reveal whether model assumptions — linearity, homoscedasticity (constant error variance), and normally distributed residuals — actually hold, and where a model is systematically wrong. Four tools are central: residual plots, prediction-error plots, alpha-selection plots, and Cook's distance.

Residual plots

A residual plot graphs the residuals (actual value minus predicted value) against the predicted values or an input feature. Random scatter around zero indicates a good fit; a funnel shape indicates heteroscedasticity — error variance that grows or shrinks with the fitted value; and a curved pattern indicates the model is missing a non-linear relationship present in the data.

Prediction-error plots

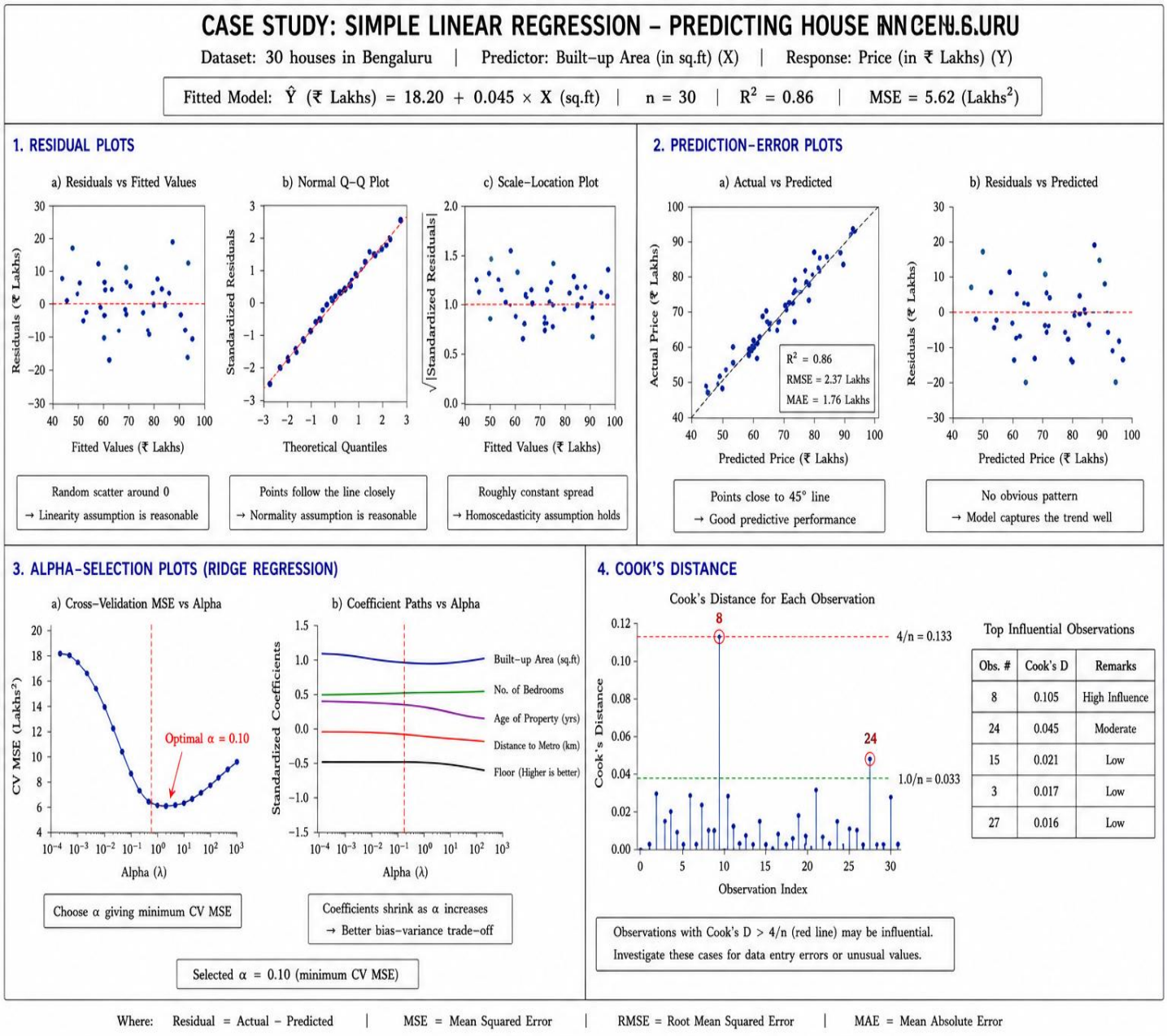
A prediction-error plot is a scatter of actual vs. predicted values, with a 45° reference line ($y = x$) representing perfect prediction. Points lying close to the line indicate accurate predictions; systematic deviation from the line in a specific value range reveals exactly where the model is weakest — for example, under-prediction of very expensive houses. This is a far more diagnostic tool than a single aggregate metric like R^2, because it shows where and how much the model errs, not just how much on average.

Alpha-selection plots (Ridge/Lasso)

For regularized regression, an alpha-selection plot shows how coefficients (and validation error) change as the regularization parameter α varies. It helps select an α that balances bias and variance while keeping coefficients interpretable and non-degenerate. For Lasso specifically, the plot shows coefficients being driven to exactly zero as α increases — directly relevant to interpretability, since it reveals which features the model considers dispensable.

Cook's distance

Cook's distance measures how much removing a single observation would change the fitted regression model — in effect, the influence of each individual point on the model's coefficients. High Cook's distance values flag potential outliers or high-leverage points that disproportionately affect the fit. This matters for explainability, since an unexplainable prediction may simply be driven by one anomalous training point rather than a genuine pattern.

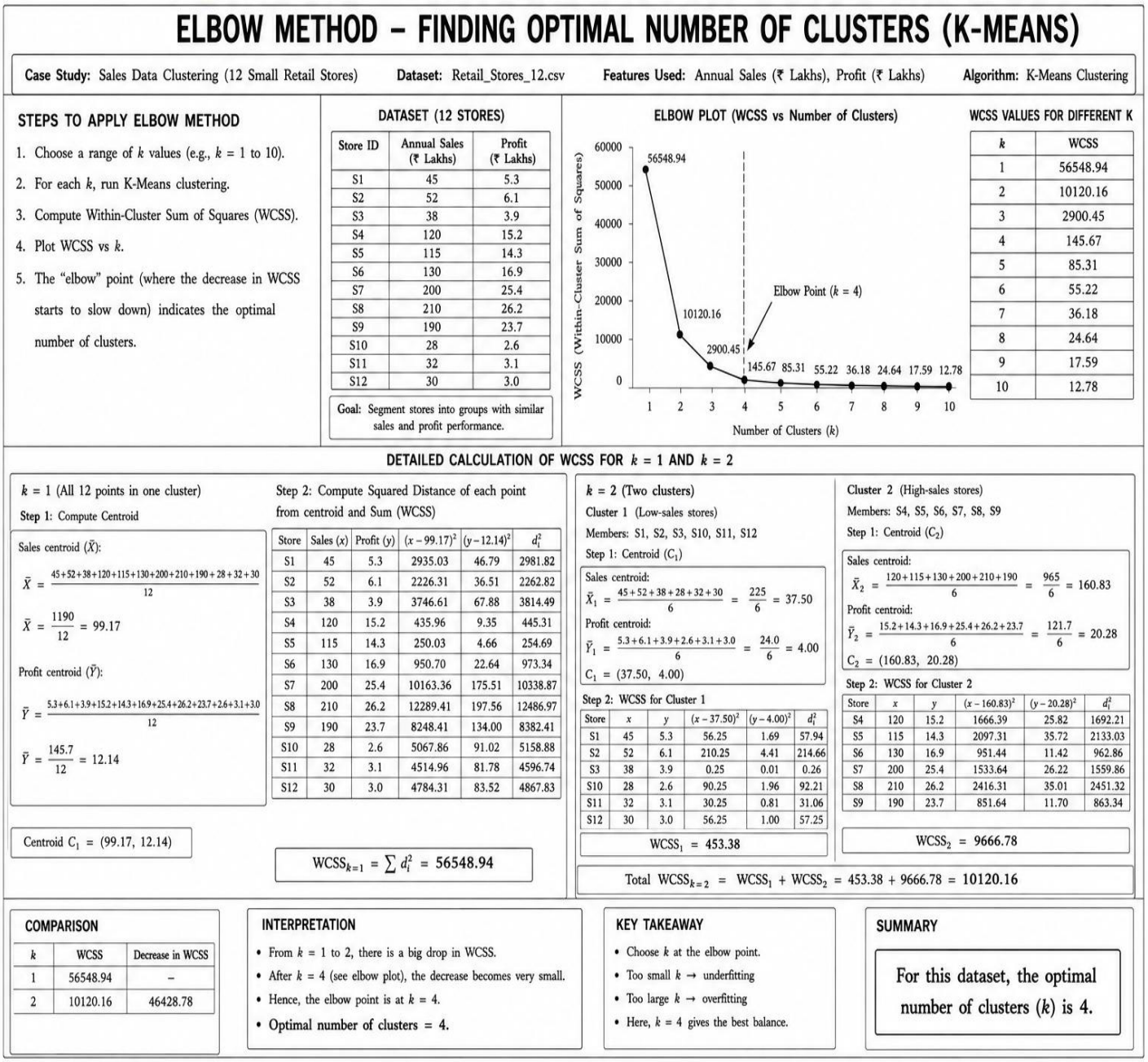


Clustering Model Visualization

Unsupervised models lack ground-truth labels, which makes visualization essential for validating and explaining cluster structure rather than relying on a single accuracy-like metric. Three tools are central: the elbow method, the silhouette coefficient, and inter cluster distance maps.

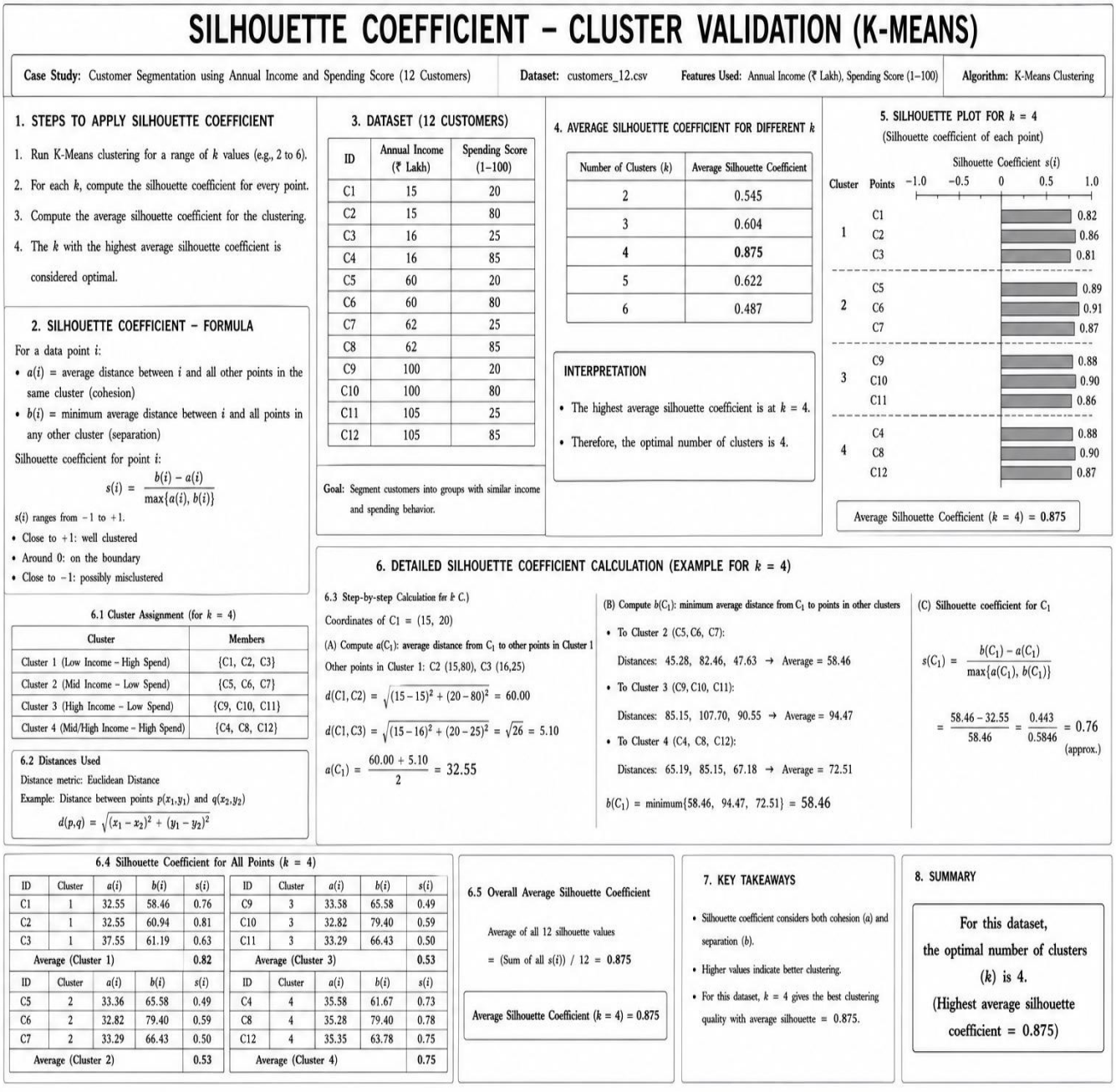
Elbow method

The elbow method runs K-Means (or a similar algorithm) for a range of k values, plotting within-cluster sum of squares (WCSS, or inertia) against k. WCSS always decreases as k increases, but the rate of decrease slows sharply beyond a good choice of k, creating a visible 'elbow' in the plot — a simple, visual heuristic for selecting the number of clusters, though the elbow can sometimes be ambiguous.



Silhouette coefficient

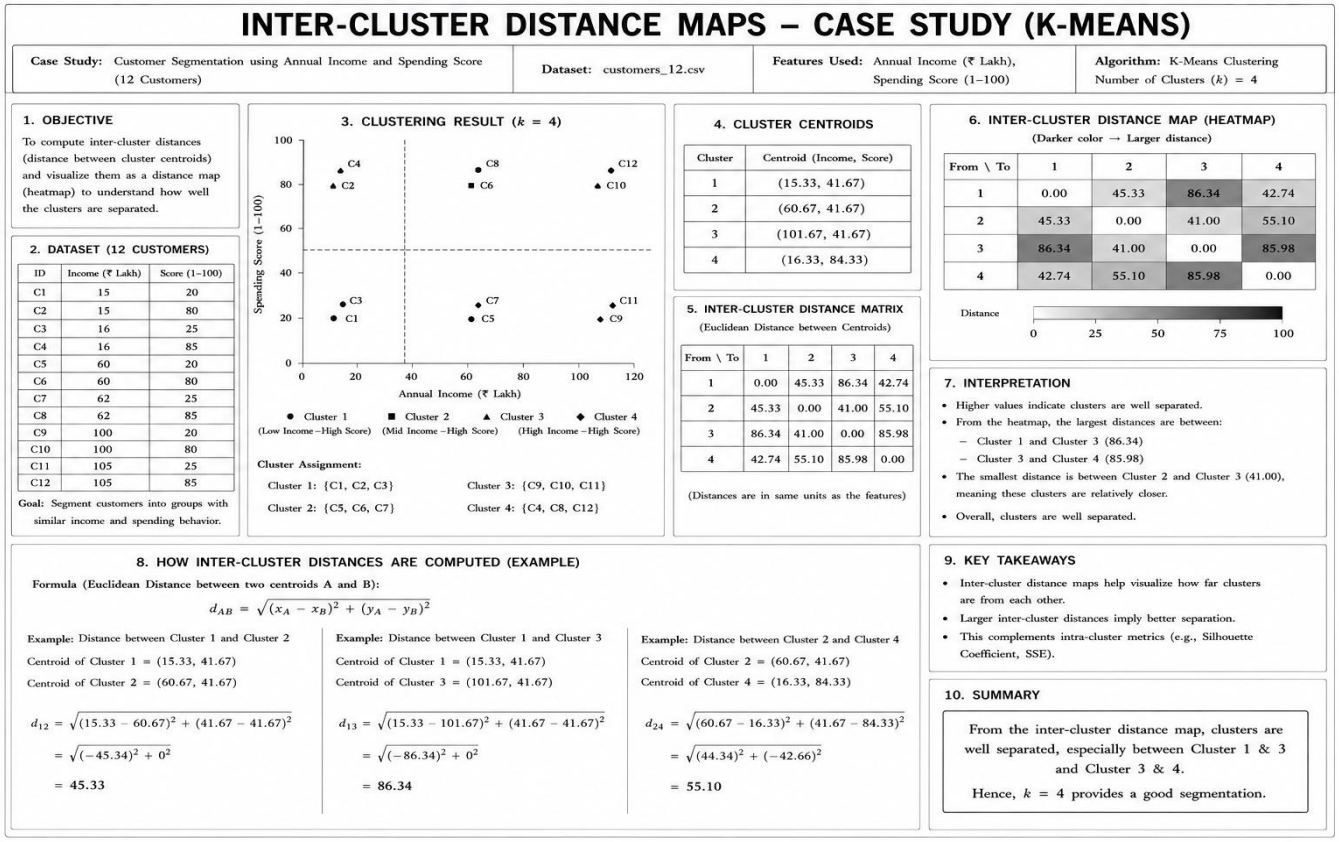
The silhouette coefficient measures, for each data point, how similar it is to points in its own cluster (average distance a) compared to points in the next-nearest cluster (average distance b): s = (b − a) / max(a, b), ranging from −1 to +1. Higher values indicate well-separated, cohesive clusters; values near zero or negative indicate poor or ambiguous assignment. Plotted per cluster as horizontal bars, wide and uniformly high bars indicate good separation, while thin or negative bars flag overlapping or misassigned points — a more precise, per-cluster quality check than the elbow method's single inertia curve.



Inter cluster distance maps

Inter cluster distance maps project cluster centers into 2D space (e.g. via multidimensional scaling, MDS), with circle size proportional to cluster membership and inter-circle distance proportional to how different the clusters are. This gives an intuitive, global picture of how well-separated clusters are and which clusters are most similar to each other — valuable for

explaining the overall clustering structure to non-technical stakeholders without requiring them to interpret raw distance numbers.

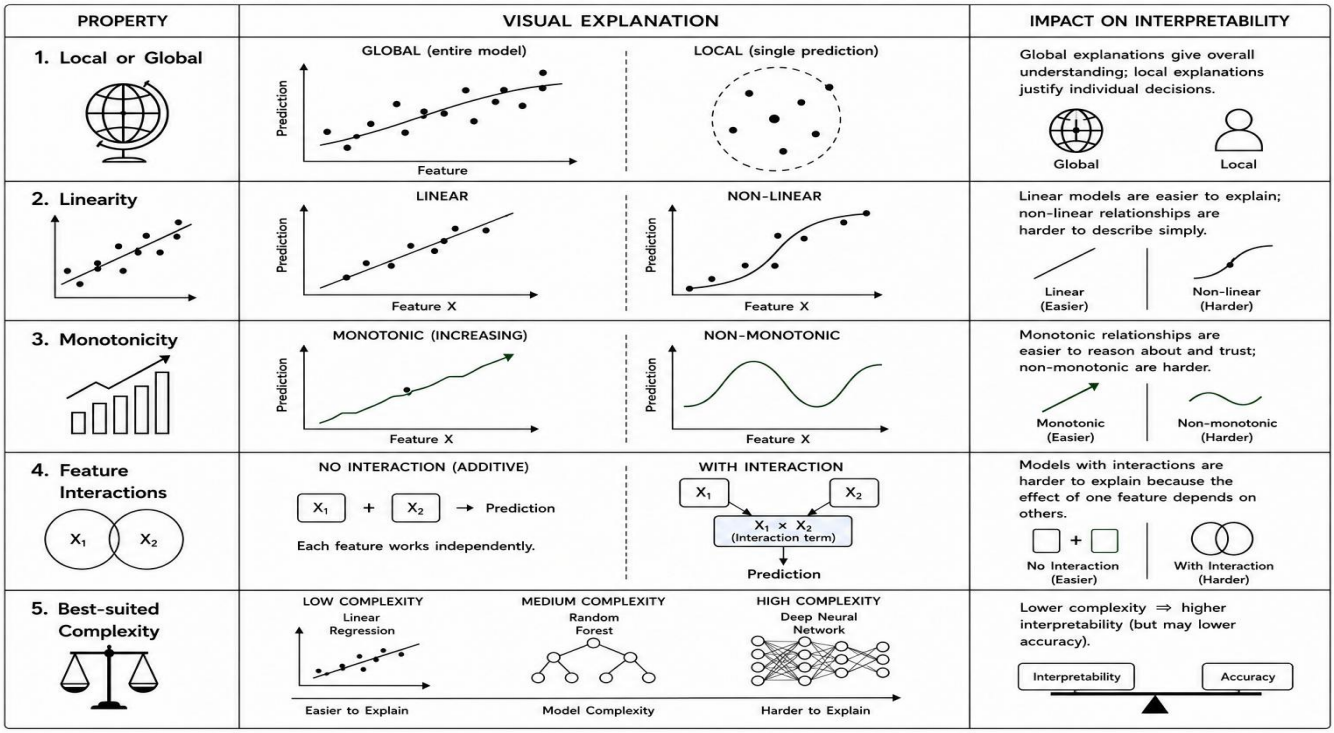


Properties of Interpretable Machine-Learning Models

Rather than a vague 'black-box vs. white-box' label, interpretability can be broken down into a small set of structured properties that let different models be compared on the same terms. Intrinsic interpretability specifically refers to transparency that comes directly from a model's own structure — its equation, its tree splits, its rules — requiring no separate, external explanation technique. This differs from model visualization, which uses external plots and diagnostic charts (confusion matrices, ROC curves, residual plots, and so on) to evaluate how well any model performs, including black-box models; visualization explains performance, while intrinsic interpretability explains reasoning.

The five key properties

- **Local or Global:** whether an explanation applies to the entire model's behaviour (global) or to a single specific prediction (local).
- **Linearity:** whether the relationship between a feature and the prediction is a straight line — a constant change in output per unit change in input.
- **Monotonicity:** whether increasing a feature's value always moves the prediction in one consistent direction, never reversing.
- **Feature Interactions:** whether the model captures combined effects between two or more features, or treats every feature's contribution independently.
- **Best-suited Complexity:** the overall complexity of the model, which governs how easy it is to explain — lower complexity generally means higher interpretability, though it may trade off some accuracy.



How traditional algorithms satisfy these properties

Algorithm	Local/Global	Linear?	Monotonic?	Interactions?	Complexity
Linear Regression	Global	Yes	Yes	No	Low
Logistic Regression	Global	Linear (in log-odds)	Yes	No	Low
Decision Tree (shallow)	Global (whole tree) & Local (one path)	No	Not guaranteed	Yes, implicitly	Low–Medium
Naive Bayes	Global	No (probabilistic)	Not guaranteed	No (assumes independence)	Low
Rule Induction	Local (per rule) & Global (rule set)	No	No	Yes, within a rule	Low

Traditional ML algorithms Interpretability:

The traditional interpretable algorithms include Linear Regression, Logistic Regression, Decision Trees, Naive Bayes, GLMs, GAMs, Bayesian Networks, and Rule Induction. Each is inherently transparent by construction — its own structure is the explanation, with no need for a separate post-hoc technique.

1.Linear Regression

Linear regression models the target as a weighted sum of the input features plus an intercept: $y = \beta0 + \beta1x1 + \beta2x2 + ... + \beta nxn$. Each weight (β) directly represents how much the prediction changes for a one-unit increase in that feature, making the model transparent by construction.

Interpretability Properties

Property	How it applies to this model
Local / Global	Global — the same equation and the same set of weights explain every prediction the model makes, not just one instance.
Linear	Yes — by definition. The prediction changes by exactly β_j for every one-unit increase in feature x_j, everywhere in the feature space.
Monotonic	Yes — since the relationship is a straight line, each feature's effect only ever pushes the prediction in one direction (up if $\beta_j > 0$, down if $\beta_j < 0$).
Feature Interactions	No — features are assumed to act independently. Interactions can only appear if the modeller manually adds an interaction term such as $x1 \times x2$.
Complexity	Low — the entire model is summarized by n+1 numbers (one weight per feature plus the intercept).

Worked Example: Predicting House Price

Model:

$$\text{House Price} = 5,00,000 + 4,000(\text{Size in sq.ft.}) + 2,00,000(\text{Bedrooms})$$

Interpretation:

- For every additional **1 sq.ft.**, the predicted house price increases by **₹4,000**, keeping the number of bedrooms constant.
- For every additional **bedroom**, the predicted price increases by **₹2,00,000**, keeping the house size constant.
- These effects are **constant and additive** in the linear regression model.

Example:

For a **1,500 sq.ft., 3-bedroom house**:

$$\begin{aligned}\text{Price} &= 5,00,000 + 4,000(1,500) + 2,00,000(3) \\ &= 5,00,000 + 60,00,000 + 6,00,000 \\ &= \boxed{₹71,00,000}\end{aligned}$$

Linear regression is the benchmark for interpretability: global, linear, monotonic, no automatic interactions, and minimal complexity.

2. Logistic Regression

Logistic regression predicts the probability of a binary outcome by applying the logistic (sigmoid) function to a weighted sum of the features. The weighted sum is linear, but it models the log-odds of the outcome rather than the outcome itself: $\log(p / (1-p)) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$.

Interpretability Properties

Property	How it applies to this model
Local / Global	Global — one equation and one set of weights describes the log-odds of the outcome for every possible input.
Linear	Linear in the log-odds (logit) space, not in the raw probability space. A one-unit increase in x_j changes the log-odds by exactly β_j, but the change in probability itself is not constant.
Monotonic	Yes — because the logistic function is itself monotonic, a positive β_j means increasing x_j always increases the predicted probability, and vice-versa for negative β_j.
Feature Interactions	No — as with linear regression, interactions are not captured unless explicitly added as extra terms.
Complexity	Low — again just n+1 weights, though effects are usually reported as odds ratios (e^β) for easier interpretation.

Worked Example: Predicting Loan Default

Model

$$\boxed{\log\left(\frac{P(D=1)}{1-P(D=1)}\right) = -3.2 + 0.9M - 0.02C}$$

M = missed payments, C = credit score.

Example

For **2 missed payments** and **credit score = 650**:

$$\begin{aligned}\text{Log-odds} &= -3.2 + 0.9(2) - 0.02(650) \\ &= -14.4\end{aligned}$$

Convert to probability:

$$P(D=1) = \frac{1}{1+e^{14.4}} \approx 0.00000056$$

$$\boxed{P(\text{Default}) \approx 0.000056\%}$$

Interpretation

- **Missed payments (+0.9):** increase default risk.
- **Credit score (−0.02):** higher score decreases default risk.
- The coefficients make the model **directly interpretable**.

Logistic regression keeps all the interpretability of linear regression but shifts the linear/monotonic guarantee to the log-odds scale instead of the raw probability scale.

3. Generalized Linear Model (GLM)

A Generalized Linear Model extends ordinary linear regression to response variables that are not normally distributed, by combining three components: a linear predictor ($\eta = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$), a chosen probability distribution for the response from the exponential family (Normal, Binomial, Poisson, Gamma, and others), and a link function $g(\cdot)$ that connects the linear predictor to the expected value of the response: $g(E[y]) = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$.

Link functions

- **Identity link, g(μ) = μ:** used for a normally distributed response — recovers ordinary Linear Regression.
- **Logit link, g(μ) = ln(μ / (1−μ)):** used for a Binomial (binary) response — recovers Logistic Regression.
- **Log link, g(μ) = ln(μ):** used for a Poisson (count) response — recovers Poisson Regression.

In every case, the link function is what lets a strictly linear combination of features be mapped onto an appropriate output range — any real number, a probability between 0 and 1, or a non-negative count — while the model still retains coefficient-based interpretability.

Linear, Logistic and Poisson Regression as special cases of GLM

Regression type	Response type	Link function	Interpretation of e^β
Linear Regression	Continuous, real-valued	Identity: g(μ) = μ	Not applicable — β is the direct unit change in y
Logistic Regression	Binary (0/1)	Logit: g(μ) = ln(μ/(1−μ))	e^β = odds ratio for a one-unit increase in x
Poisson Regression	Count (0, 1, 2, ...)	Log: g(μ) = ln(μ)	e^β = multiplicative effect on expected count

Interpretability Properties

Property	How it applies to this model
Local / Global	Global — one linear predictor and one set of coefficients apply across the whole dataset.
Linear	Linear only on the scale of the link function (e.g., log-scale for Poisson GLM), not on the original output scale, since the link function itself is a non-linear transform.
Monotonic	Yes — standard link functions (log, logit, inverse) are monotonic, so each feature's effect on the linear predictor translates into a consistently-directioned effect on the actual prediction.
Feature Interactions	No — same as linear/logistic regression, interactions must be added manually as product terms.
Complexity	Low to medium — still just one coefficient per feature, but interpretation additionally requires knowing which link function and distribution family were used.

Worked Example: Predicting Hospital Visit Counts

Model

$$\boxed{\log(E[V]) = 0.4 + 0.15A + 0.5C}$$

where:

- V = number of hospital visits
- A = age in decades
- C = chronic condition (1 = Yes, 0 = No)

Example

For a **60-year-old patient with a chronic condition**:

$$A = 6, \qquad C = 1$$

$$\log(E[V]) = 0.4 + 0.15(6) + 0.5(1)$$
$$= 1.8$$

Convert to expected visits:

$$E[V] = e^{1.8} \approx \boxed{6.05}$$

Interpretation

- A chronic condition has coefficient **+0.5**.
- Expected visits are multiplied by:

$$e^{0.5} \approx 1.65$$

- Thus, having a chronic condition is associated with a **65% increase in expected visits**, holding age constant.

GLMs extend linear regression's interpretability to a much wider range of outcome types by wrapping the linear predictor inside a monotonic link function.

4. Generalized Additive Model (GAM)

A GAM extends the GLM by replacing each linear term β_j·x_j with a flexible, learned smooth function f_j(x_j): g(E[Y]) = β0 + f1(x1) + f2(x2) + ... + fn(xn). This lets each feature have its own non-linear shape while the overall model remains additive (no automatic interactions between features).

Interpretability Properties

Property	How it applies to this model
Local / Global	Global — the shape function f_j for each feature is fixed and applies to every prediction, though it is usually inspected feature-by-feature via a plot.
Linear	No — each f_j can be curved (e.g., U-shaped, S-shaped) rather than a straight line, so the effect of a feature is no longer a constant per unit increase.
Monotonic	Not guaranteed — a shape function is free to rise and then fall (or vice-versa) unless the modeller explicitly constrains f_j to be monotonic.
Feature Interactions	No, by default — the model is purely additive (sum of individual shape functions). Pairwise interaction terms f_ij(xi, xj) can be added deliberately, at the cost of extra complexity.

Property	How it applies to this model
Complexity	Medium — instead of one number per feature, understanding the model means inspecting one curve (shape function) per feature.

Worked Example: Predicting Bike Rentals

Model

$$\boxed{E[\text{Rentals}] = f_1(\text{Temperature}) + f_2(\text{Humidity}) + f_3(\text{Day of Week})}$$

Example

Suppose we examine how each feature affects the expected number of bike rentals.

Temperature:

$$0^\circ C \rightarrow 25^\circ C$$

Rentals increase as temperature rises, but after about **30°C**, rentals begin to decrease.

Humidity:

As humidity increases beyond **70%**, expected rentals decrease.

Interpretation

- $f_1(\text{Temperature})$ captures the **non-linear effect of temperature**.
- $f_2(\text{Humidity})$ captures the **effect of humidity independently**.
- $f_3(\text{Day of Week})$ captures differences between days.

Thus, **each feature has its own interpretable curve**, allowing non-linear relationships to be understood without using a single straight-line coefficient.

GAMs trade the strict linearity of GLMs for flexible, feature-specific curves, while keeping the model additive and each feature's effect visualizable on its own.

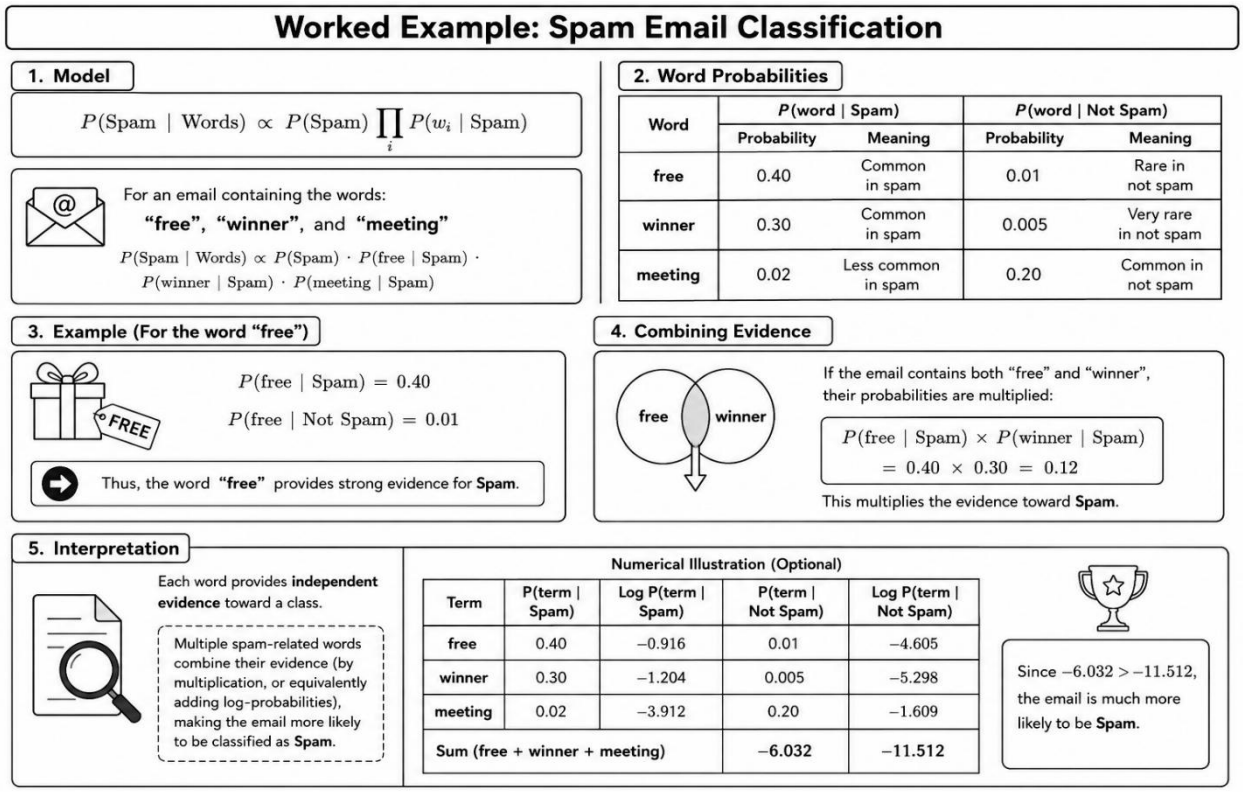
5. Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the 'naive' assumption that all features are conditionally independent given the class label: P(class | x1,...,xn) ∝ P(class) × Π P(xi | class). Because of this independence assumption, taking logs turns the model into a simple sum of per-feature log-probabilities.

Interpretability Properties

Property	How it applies to this model
Local / Global	Global — the per-feature conditional probability tables/distributions are learned once and reused for every prediction.
Linear	Effectively linear/additive on the log-probability (log-odds) scale, since the class-conditional independence assumption turns the joint probability into a sum of individual log terms.
Monotonic	Yes, with respect to each feature's own log-likelihood ratio — increasing evidence for a class from one feature does not get reversed by that same feature's other values (each feature contributes independently).

Property	How it applies to this model
Feature Interactions	No — this is the central assumption of the model: features are conditionally independent given the class, so no interactions between features are captured at all.
Complexity	Low — only requires one probability distribution per feature per class, which scales linearly with the number of features.



Naive Bayes is simple and fast to interpret because each feature votes independently — but that same independence assumption means it cannot represent real feature interactions.

6. Bayesian Network

A Bayesian Network represents variables as nodes in a directed acyclic graph (DAG), with edges encoding direct probabilistic dependencies and each node storing a conditional probability table (CPT) given its parents. Unlike Naive Bayes, it does not assume all features are independent — it explicitly models which variables depend on which.

Interpretability Properties

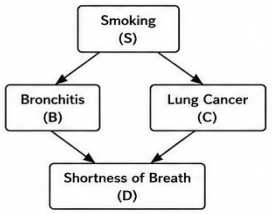
Property	How it applies to this model
Local / Global	Both — the graph structure and CPTs are global (fixed for the whole model), but explanations for a specific prediction can be traced locally along the specific path of nodes that influenced it.
Linear	No — conditional probability tables can encode arbitrary, non-linear relationships between a node and its parent nodes.

Property	How it applies to this model
Monotonic	Not guaranteed — a CPT can represent any pattern of increasing/decreasing probability as a parent variable changes, unless deliberately constrained to be monotonic.
Feature Interactions	Yes — this is the network's defining strength. Edges explicitly encode how two or more variables jointly influence another variable (e.g., a node with two parents naturally combines both parents' effects).
Complexity	Medium to high — grows with the number of nodes, edges, and the size of each node's conditional probability table (which grows exponentially with the number of parents).

Worked Example: Bayesian Network – Medical Diagnosis

1. Network Structure

Smoking can cause Bronchitis and Lung Cancer.
Bronchitis and Lung Cancer together can cause Shortness of Breath.



Variables (all binary)

S : Smoking (Yes, No)
B : Bronchitis (Yes, No)
C : Lung Cancer (Yes, No)
D : Shortness of Breath (Yes, No)

2. Conditional Probability Tables (CPTs)

P(S)

S	P(S)
Yes	0.30
No	0.70

P(B | S)

S	B	P(B | S)
Yes	Yes	0.60
Yes	No	0.40
No	Yes	0.10
No	No	0.90

P(C | S)

S	C	P(C | S)
Yes	Yes	0.10
Yes	No	0.90
No	Yes	0.01
No	No	0.99

P(D | B, C)

B	C	D	P(D | B, C)
Yes	Yes	Yes	0.95
Yes	Yes	No	0.05
Yes	No	Yes	0.60
Yes	No	No	0.40
No	Yes	Yes	0.70
No	Yes	No	0.30
No	No	Yes	0.05
No	No	No	0.95

3. Case / Evidence

We observe that the patient has Shortness of Breath = Yes.

Evidence: $D = Yes$

4. Goal

We want to estimate the probability that the patient is a smoker given the evidence:

$$P(S = Yes \mid D = Yes) = ?$$

5. Inference (Using Enumeration)

We compute $P(S = Yes, D = Yes)$ and $P(S = No, D = Yes)$ by summing over all possible values of B and C.

$$P(S, D) = \sum_B \sum_C P(S)P(B|S)P(C|S)P(D|B, C)$$

6. Calculation

When S = Yes ($P(S{=}Yes, D{=}Yes)$)

B	C	P(B|Yes)	P(C|Yes)	P(D=Yes|B,C)	Product
Yes	Yes	0.60	0.10	0.95	0.0570
Yes	No	0.60	0.90	0.60	0.3240
No	Yes	0.40	0.10	0.70	0.0280
No	No	0.40	0.90	0.05	0.0180

Sum = 0.4270 ⇒ $P(S = Yes, D{=}Yes) = 0.30 \times 0.4270 = 0.1281$

When S = No ($P(S{=}No, D{=}Yes)$)

B	C	P(B|No)	P(C|No)	P(D=Yes|B,C)	Product
Yes	Yes	0.10	0.01	0.95	0.00095
Yes	No	0.10	0.99	0.60	0.05940
No	Yes	0.90	0.01	0.70	0.00630
No	No	0.90	0.99	0.05	0.04455

Sum = 0.11120 ⇒ $P(S{=}No, D{=}Yes) = 0.70 \times 0.11120 = 0.07784$

7. Result

Normalize to get the posterior probability.

$$P(D = Yes) = 0.1281 + 0.07784 = 0.20594$$

$$P(S = Yes \mid D = Yes) = \frac{0.1281}{0.20594} = 0.6217$$

$$P(S = No \mid D = Yes) = 1 - 0.6217 = 0.3783$$

Interpretation: Given that the patient has Shortness of Breath, the probability that the patient smokes is about **62.2%**.

Bayesian Networks trade some simplicity for the ability to explicitly model dependencies and interactions between variables through their graph structure.

Comparing the two

Aspect	Naive Bayes	Bayesian Networks
Independence assumption	All features conditionally independent given the class	Only non-descendants independent, given parents (edges model real dependencies)
Representation	Prior + per-feature conditional probabilities (no graph)	Directed Acyclic Graph (DAG) of variables + local CPTs
Explanation form	Additive/multiplicative per-feature contributions	Visual graph traversal + inference (belief propagation) trail
Captures feature interactions?	No	Yes, explicitly via edges

Typical strength	Fast, simple, works well for text/high-dimensional data	Models causal/probabilistic structure among variables explicitly

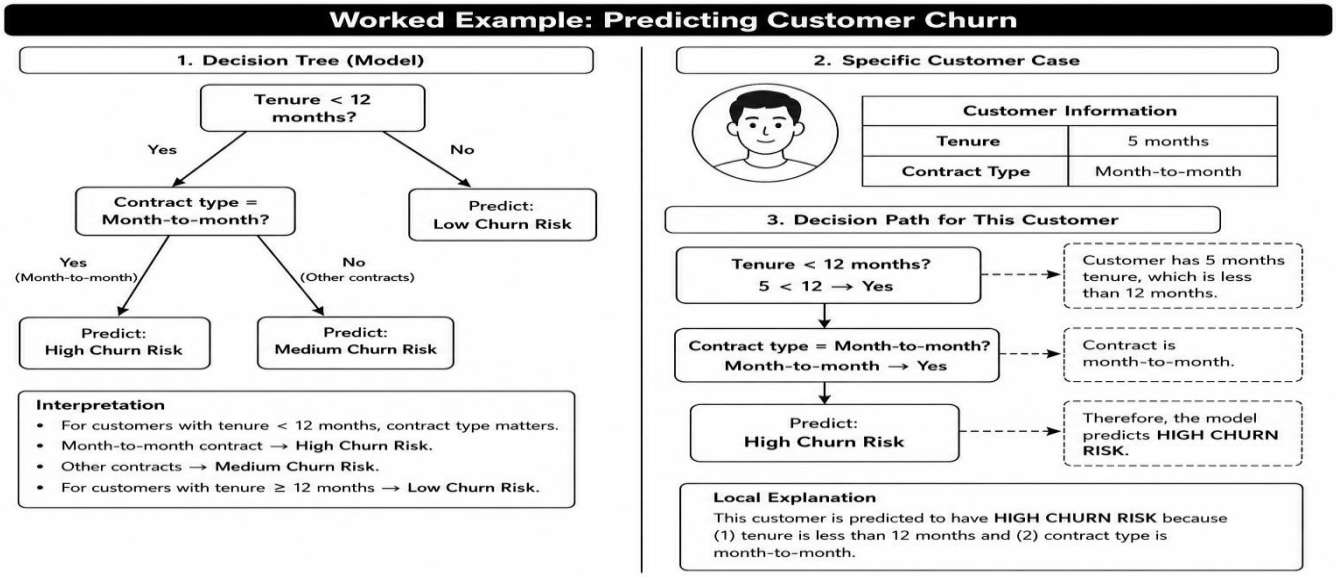
Naive Bayes trades away feature-interaction modelling for simplicity and speed, explaining decisions as independent per-feature contributions; Bayesian Networks keep dependencies explicit in a graph, offering a richer but more complex explanation that can be traced through the network structure itself.

7. Decision Tree

A decision tree recursively splits the feature space using a sequence of if-then conditions on individual features, forming a tree of decision nodes that ends in leaf nodes holding the final prediction. A prediction for any instance is read as the single path from the root to a leaf.

Interpretability Properties

Property	How it applies to this model
Local / Global	Both — the full tree is a global model, but any single prediction has a natural local explanation: the specific chain of if-then splits (the root-to-leaf path) used for that instance.
Linear	No — the tree produces a piecewise-constant, step-like function; predictions jump abruptly at each split threshold rather than changing smoothly.
Monotonic	Not guaranteed globally — a feature can be split on at multiple points in different directions, so the overall trend for a feature is not necessarily one-directional (though a single, well-behaved tree can be monotonic if the modeller enforces it).
Feature Interactions	Yes, automatically — because each split is conditioned on the outcome of previous splits, the tree naturally captures interactions (e.g., a feature's effect at one leaf can be different from its effect at another leaf, depending on prior splits).
Complexity	Depends directly on tree size — a shallow tree (few splits) is very easy to read; a deep tree with many leaves becomes as hard to interpret as a black box.



Decision trees offer intuitive, rule-like local explanations and free feature interactions, but interpretability degrades quickly as depth and leaf count grow.

8. Rule Induction (Decision Rules)

Rule induction learns a set of human-readable IF-THEN rules directly from data (e.g., via algorithms like RIPPER, OneR, or RuleFit), where each rule combines one or more feature conditions with AND, and predicts an outcome if all its conditions are satisfied. Instances not covered by any specific rule usually fall back to a default prediction.

Interpretability Properties

Property	How it applies to this model
Local / Global	Primarily local — each individual rule only explains the subset of instances that satisfy its conditions; the complete rule set together forms the global model.
Linear	No — rules define hard, non-linear regions (boxes/subsets) of the feature space rather than a continuous linear relationship.
Monotonic	Not guaranteed — different rules can point to different outcomes for increasing values of the same feature, depending on which other conditions are combined with it.
Feature Interactions	Yes — a rule's AND-combination of multiple conditions is itself an explicit, human-readable interaction between those features (e.g., 'income > 50k AND credit_score > 700').
Complexity	Depends on the number of rules and the number of conditions per rule — a handful of short rules is very readable; hundreds of long rules approach black-box complexity.

Worked Example: Credit Card Approval Rules

1. Rule Set (Model)

Rule 1:
IF income > 50,000 **AND** credit_score > 700
THEN approve.

Rule 2:
IF existing_debt > 20,000 **AND** credit_score < 600
THEN reject.

Default:
IF no rule matches
THEN refer for manual review.

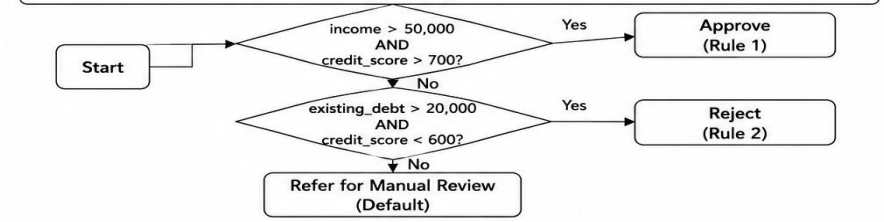
Interpretation

- **Rule 1** explicitly captures the interaction between income and **credit score** — high income alone (without a good credit score) does not trigger approval.
- **Rule 2** captures a different interaction — high existing **debt** together with **low credit score** leads to rejection.
- If neither rule matches, the case is referred for manual review.

2. Example Customer Cases

Case	Income (₹)	Credit Score	Existing Debt (₹)	Decision
A	60,000	720	10,000	Approve (Rule 1)
B	60,000	580	10,000	Refer for Review (No rule matches)
C	40,000	550	25,000	Reject (Rule 2)
D	55,000	680	22,000	Refer for Review (No rule matches)

3. How the Rules Work (Decision Flow)



Explanation for a Specific Case (Case A)

- income = 60,000 (> 50,000) → True
- credit_score = 720 (> 700) → True

Both conditions are True, so the customer follows the path: Start → Rule 1 → Approve.

Rule induction produces some of the most human-readable explanations of any model, with feature interactions built directly into each rule's conditions — at the cost of coverage and consistency across the whole feature space.

Structural comparison

Aspect	Decision Tree	Rule Induction
Structure	Single hierarchical tree, shared splits across branches	Flat, independent set of IF-THEN rules
Prediction path	One root-to-leaf path per input	One (or more) matching rule per input
Explanation unit	A path = conjunction of conditions along one branch	A rule = conjunction of conditions, self-contained
Modularity	Branches share ancestor conditions — not independent	Each rule is fully independent — can add/remove freely
Typical output form	Class/value at the leaf	Class/value stated directly in the rule's THEN part

The key structural difference is that a tree's explanation is embedded in a shared hierarchy, while rule induction produces fully modular, independent statements that can each be verified in isolation — two different routes to the same goal of intrinsic, structure-based interpretability.

Short answer questions:

1.What is the purpose of model validation?

2.Differentiate between training data and test data.

3.Define hyperparameter with one example.

4.Name any two hyperparameter tuning techniques.

5.What is a validation curve?

6.What is a learning curve?

7.Define ROC curve and AUC.

8.When is a Precision–Recall Curve preferred over an ROC curve?

9.What is Cook’s distance used for?

10.List any four properties of interpretable machine-learning models.

11.What is the main difference between model visualization and intrinsic interpretability?

Long answer questions:

1.Explain the roles of training, validation and test data in model development.

2.Explain how Linear Regression provides an interpretable prediction using its coefficients.

3.Explain how Decision Trees and Rule Induction provide interpretable predictions.

4.Explain model validation, hyperparameter tuning and model selection. Discuss the role of cross-validation in selecting a suitable model.

5.Explain validation curves and learning curves with suitable examples. Show how they help identify overfitting and underfitting.

6.Explain the major visualization techniques used for classification models, including confusion matrix, ROC–AUC, Precision–Recall Curve and discrimination threshold plots.

7.Explain the major visualization techniques used for regression models, including residual plots, prediction-error plots, alpha-selection plots and Cook's distance.

8.Explain the elbow method, silhouette coefficient and intercluster distance maps for evaluating and interpreting clustering models.

9.Explain the important properties of interpretable machine-learning models. Illustrate these properties using traditional interpretable algorithms.

10.Explain Generalized Linear Models (GLMs). Discuss their link functions and explain how Linear, Logistic and Poisson Regression fit within the GLM framework.

11.Explain Generalized Additive Models (GAMs). How do GAMs provide non-linear relationships while maintaining feature-wise interpretability?

12. Explain the interpretability of Naive Bayes and Bayesian Networks.